Tutorial: Filtering Microarray Data with a Simple Python Script

This tutorial explains how to write, understand, and run a **minimal Python script** that filters a microarray data file based on log2-transformed sample values. The tutorial assumes you already have a working IDE (for example, Thonny, PyCharm, or VS Code).

---

1. Problem Overview

We start with a raw text file (`raw_data.txt`) that contains microarray data with at least these columns:

- Probe
- Name
- Chromosome
- Position
- Feature
- Sample A data
- Sample B data

We want to keep only rows that meet **two biological and mathematical criteria**:

1. **At least one strong signal** The log2 value of either sample is greater than 2:

   `log2(SampleA) > 2  OR  log2(SampleB) > 2`

   This means that at least one of the samples has a value greater than 4 in raw scale.

2. **Large difference between samples** The absolute difference in log2 values is greater than 3:

   `|log2(SampleA) - log2(SampleB)| > 3`

   This means the two samples differ by more than 8-fold (since $2^3 = 8$).

The script will output a new file (`filtered_data.txt`) containing only the rows that satisfy both conditions.

Additionally, it will add **two new columns**: - `log2_SampleA` - `log2_SampleB`

---

2. The Script

Create a file called **`filter_microarray_simple.py`** and paste the following code inside it:

```python
#!/usr/bin/env python3
# Simple microarray filter using only sys.argv and basic string operations (no csv module).
# Usage:
#     python filter_microarray_simple.py input.txt output.txt [SampleAName] [SampleBName]
#
# Rules to keep a row:
#   1) log2(SampleA) > 2 OR log2(SampleB) > 2
#   2) |log2(SampleA) - log2(SampleB)| > 3
#
# Output: same columns as input + log2_SampleA, log2_SampleB

import sys, math

def detect_delim(header_line: str) -> str:
    if "\t" in header_line:
        return "\t"
    elif "," in header_line:
        return ","
    elif ";" in header_line:
        return ";"
    else:
        return None  # fallback to splitting on whitespace

def find_sample_indices(headers, sampleA_name=None, sampleB_name=None):
    lower = [h.strip().lower() for h in headers]
    candA = [sampleA_name, "samplea", "sample a", "sample a data"]
    candB = [sampleB_name, "sampleb", "sample b", "sample b data"]
    def pick(cands):
        for c in cands:
            if c and c.strip().lower() in lower:
                return lower.index(c.strip().lower())
        return None
    idxA = pick(candA)
    idxB = pick(candB)
    if idxA is None or idxB is None:
        raise SystemExit("Could not find Sample A/B columns. Available headers: " + str(hea
    return idxA, idxB

def safe_float(x):
    try:
        return float(x)
    except Exception:
        return None

def main():
    if len(sys.argv) < 3:
```

```
    print("Usage: python filter_microarray_simple.py <input> <output> [SampleAName] [Sam
    sys.exit(2)
in_path = sys.argv[1]
out_path = sys.argv[2]
sampleA_name = sys.argv[3] if len(sys.argv) > 3 else None
sampleB_name = sys.argv[4] if len(sys.argv) > 4 else None

with open(in_path, "r", encoding="utf-8", errors="replace") as f:
    lines = f.read().splitlines()
if not lines:
    print("Empty input file.")
    sys.exit(1)

header = lines[0]
delim = detect_delim(header)
headers = header.split(delim) if delim is not None else header.split()
idxA, idxB = find_sample_indices(headers, sampleA_name, sampleB_name)

out_headers = headers + ["log2_SampleA", "log2_SampleB"]
with open(out_path, "w", encoding="utf-8") as g:
    g.write((delim if delim else "\t").join(out_headers) + "\n")

    total = 0
    kept = 0
    skipped_nonpos = 0

    for line in lines[1:]:
        if not line.strip():
            continue
        cols = line.split(delim) if delim else line.split()
        if len(cols) < len(headers):
            continue

        a = safe_float(cols[idxA])
        b = safe_float(cols[idxB])
        total += 1
        if a is None or b is None or a <= 0.0 or b <= 0.0:
            skipped_nonpos += 1
            continue

        la = math.log2(a)
        lb = math.log2(b)
        cond1 = (la > 2.0) or (lb > 2.0)
        cond2 = abs(la - lb) > 3.0
        if cond1 and cond2:
            kept += 1
```

```
                g.write((delim if delim else "\t").join(cols + [f"{la:.6f}", f"{lb:.6f}"]) +

    print(f"Processed rows (excluding header): {total}")
    print(f"Kept rows: {kept}")
    print(f"Skipped due to non-positive/invalid values: {skipped_nonpos}")

if __name__ == "__main__":
    main()
```

---

3. How the Script Works

1. **Reads command-line arguments** using `sys.argv` for input/output filenames.
2. **Detects the delimiter** (tab, comma, or semicolon).
3. **Finds column positions** for the Sample A and Sample B data.
4. **Computes log2 values** for each sample.
5. **Applies filtering conditions**:
   - log2(SampleA) > 2 or log2(SampleB) > 2
   - $|\log2(\text{SampleA}) - \log2(\text{SampleB})| > 3$
6. **Writes output file** containing:
   - All original columns
   - Two new columns: log2_SampleA and log2_SampleB
   - Only the filtered rows
7. **Prints a summary** of how many rows were processed and kept.

---

4. Running the Script

Example:

```
python filter_microarray_simple.py raw_data.txt filtered_data.txt "Sample A data" "Sample B
```

If your headers are just `SampleA` and `SampleB`:

```
python filter_microarray_simple.py raw_data.txt filtered_data.txt
```

---

5. Output

The resulting `filtered_data.txt` will: - Contain the same header as your input - Include two new columns: `log2_SampleA` and `log2_SampleB` - Contain only the filtered rows

Example output summary:

```
Processed rows (excluding header): 1999
Kept rows: 88
Skipped due to non-positive/invalid values: 0
```

---

6. Customizing

- Change the thresholds:

```
cond1 = (la > 2.0) or (lb > 2.0)
cond2 = abs(la - lb) > 3.0
```

- Handle zeros differently (use pseudocount):

```
la = math.log2(a + 1)
lb = math.log2(b + 1)
```

- Force a specific delimiter:

```
delim = "\t"
```

---

7. Key Takeaways

- Read and process text files line by line.
- Use `sys.argv` for flexible file input/output.
- Compute log2 transformations.
- Filter rows using logical conditions.
- Write a new output file with additional columns.

This script provides a simple and clear introduction to bioinformatics data filtering in Python.